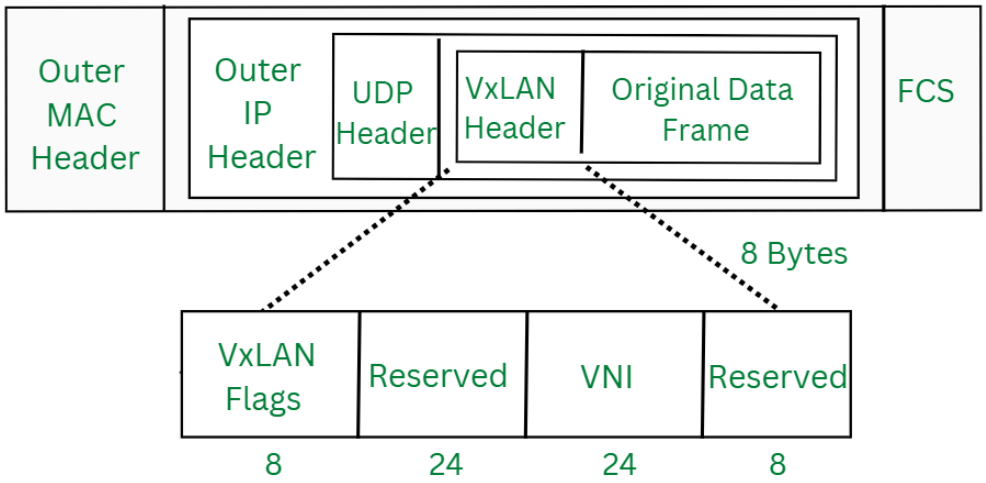
ovn的vxlan实现原理

vxlan格式



节点之间设置vxlan隧道

- 节点1

```
ovs-vsctl \
-- set Open_vSwitch . external-ids:system-id=hv1 \
-- set Open_vSwitch . external-ids:ovn-encap-type=geneve,vxlan \
-- set Open_vSwitch . external-ids:ovn-encap-ip=192.168.0.1
```

- 节点2

```
ovs-vsctl \
-- set Open_vSwitch . external-ids:system-id=hv2 \
-- set Open_vSwitch . external-ids:ovn-encap-type=vxlan \
-- set Open_vSwitch . external-ids:ovn-encap-ip=192.168.0.2
```

这样在节点1和节点2之间的vxlan隧道就已经建立了。

在ovs的open_vswitch表的external-ids:ovn-encap-type可以设置多个值。

```
/* Must be a bit-field ordered from most-preferred (higher number) to
 * least-preferred (lower number). */
enum chassis_tunnel_type {
    GENEVE = 1 << 2,
    STT    = 1 << 1,
    VXLAN  = 1 << 0
};


static struct sbrec_encap *
preferred_encap(const struct sbrec_chassis *chassis_rec,
                const struct sbrec_chassis *this_chassis)
{
    struct sbrec_encap *best_encap = NULL;
    uint32_t best_type = 0;

    for (size_t i = 0; i < chassis_rec->n_encaps; i++) {
        uint32_t tun_type = get_tunnel_type(chassis_rec->encaps[i]->type);
        if (tun_type > best_type && chassis_has_type(this_chassis, tun_type)) {
            best_type = tun_type;
            best_encap = chassis_rec->encaps[i];
        }
    }

    return best_encap;
}
具体查看：<https://github.com/ovn-org/ovn/commit/258b6f121177423155fdc32448a021207d14dfc5>
```

- 这里优先级为：GENEVE > STT > VXLAN
- preferred_encap：找两个chassis之间相同tunnel type且优先级最高的encap。

ovs中隧道port和interface的创建

```
encaps_run
 |- chassis_tunnel_add
  |- struct sbrec_encap *encap = preferred_encap(chassis_rec, this_chassis);
   |- tunnel_add
    |- struct ovsrec_interface *iface = ovsrec_interface_insert(tc->ovs_txn);
    |- struct ovsrec_port *port = ovsrec_port_insert(tc->ovs_txn);
    |- ovsrec_bridge_update_ports_addvalue(tc->br_int, port);
```

ovn设置隧道封装的openflow流表（put_encapsulation函数）

```
static void
put_encapsulation(enum mf_field_id mff_ovn_geneve,
                  const struct chassis_tunnel *tun,
                  const struct sbrec_datapath_binding *datapath,
                  uint16_t outport, bool is_ramp_switch,
                  struct ofpbuf *ofpacts)
{
    if (tun->type == GENEVE) {
        put_load(datapath->tunnel_key, MFF_TUN_ID, 0, 24, ofpacts);  //将 datapath->tunnel_key（64 位）加载到 MFF_TUN_ID 的低 24 位。如：load:0x1f->NXM_
        put_load(outport, mff_ovn_geneve, 0, 32, ofpacts); //将 outport（16 位）加载到 mff_ovn_geneve 的低 32 位，所以只会占用NXM_NX_TUN_METADATA0[0..1
        put_move(MFF_LOG_INPORT, 0, mff_ovn_geneve, 16, 15, ofpacts); //将 MFF_LOG_INPORT（0-14位）加载到 mff_ovn_geneve 的16-30位。如：move:NXM
// mff_ovn_geneve 字段存储了输出端口 (outport) 和输入端口 (MFF_LOG_INPORT)，分别占用低 16 位和高 15 位。
// MFF_TUN_ID 存储 24 位的逻辑数据路径标识符，符合 OVN 的元数据规范。
```

```
    } else if (tun->type == STT) {
        put_load(datapath->tunnel_key | ((uint64_t) outport << 24),
                 MFF_TUN_ID, 0, 64, ofpacts); // 将 datapath->tunnel_key（64 位）与 outport（16 位，左移 24 位后作为高位）组合成一个 64 位值，加载到 MFF_TU
        put_move(MFF_LOG_INPORT, 0, MFF_TUN_ID, 40, 15, ofpacts); // 将 MFF_LOG_INPORT（15 位逻辑输入端口标识符）移动到 MFF_TUN_ID 的第 40 到
// 这里的实现将 datapath->tunnel_key（24 位）放在低位，outport（16 位）放在 24 到 39 位，MFF_LOG_INPORT（15 位）放在 40 到 54 位。
    } else if (tun->type == VXLAN) {
        uint64_t vni = datapath->tunnel_key;
        if (!is_ramp_switch) {
            /* Map southbound 16-bit port key to limited 12-bit range
             * available for VXLAN, which differs for multicast groups. */
            vni |= get_vxlan_port_key(outport) << 12;
        }
        put_load(vni, MFF_TUN_ID, 0, 24, ofpacts);
    } else {
        OVS_NOT_REACHED();
    }
}

```

- 这里的mff_ovn_geneve即为tun_metadata0
- datapath->tunnel_key虽然是64位，但是应该被截断成24位，且只使用了低24位。

ovn解析隧道的openflow流表（physical_run函数）

```
    /* Table 0, priority 100.
     * ======================
     *
     * Process packets that arrive from a remote hypervisor (by matching
     * on tunnel in_port). */

    /* Add flows for Geneve, STT and VXLAN encapsulations.  Geneve and STT
     * encapsulations have metadata about the ingress and egress logical ports.
     * VXLAN encapsulations have metadata about the egress logical port only.
     * We set MFF_LOG_DATAPATH, MFF_LOG_INPORT, and MFF_LOG_OUTPORT from the
     * tunnel key data where possible, then resubmit to table 33 to handle
     * packets to the local hypervisor. */
    struct chassis_tunnel *tun;
    HMAP_FOR_EACH (tun, hmap_node, p_ctx->chassis_tunnels) {
        struct match match = MATCH_CATCHALL_INITIALIZER;
        match_set_in_port(&match, tun->ofport);

        ofpbuf_clear(&ofpacts);
        if (tun->type == GENEVE) {
            put_move(MFF_TUN_ID, 0,  MFF_LOG_DATAPATH, 0, 24, &ofpacts);
            put_move(p_ctx->mff_ovn_geneve, 16, MFF_LOG_INPORT, 0, 15,
                     &ofpacts);
            put_move(p_ctx->mff_ovn_geneve, 0, MFF_LOG_OUTPORT, 0, 16,
                     &ofpacts);
        } else if (tun->type == STT) {
            put_move(MFF_TUN_ID, 40, MFF_LOG_INPORT,   0, 15, &ofpacts);
            put_move(MFF_TUN_ID, 24, MFF_LOG_OUTPORT,  0, 16, &ofpacts);
            put_move(MFF_TUN_ID,  0, MFF_LOG_DATAPATH, 0, 24, &ofpacts);
        } else if (tun->type == VXLAN) {
            /* Add flows for non-VTEP tunnels. Split VNI into two 12-bit
             * sections and use them for datapath and outport IDs. */
            put_move(MFF_TUN_ID, 12, MFF_LOG_OUTPORT,  0, 12, &ofpacts);
            put_move(MFF_TUN_ID, 0, MFF_LOG_DATAPATH, 0, 12, &ofpacts);
        } else {
```

```
            OVS_NOT_REACHED();
        }

        put_resubmit(OFTABLE_LOCAL_OUTPUT, &ofpacts);

        ofctrl_add_flow(flow_table, OFTABLE_PHY_TO_LOG, 100, 0, &match,
                        &ofpacts, hc_uuid);
    }

    /* Add VXLAN specific rules to transform port keys
     * from 12 bits to 16 bits used elsewhere. */
    HMAP_FOR_EACH (tun, hmap_node, p_ctx->chassis_tunnels) {
        if (tun->type == VXLAN) {
            ofpbuf_clear(&ofpacts);

            struct match match = MATCH_CATCHALL_INITIALIZER;
            match_set_in_port(&match, tun->ofport);
            ovs_be64 mcast_bits = htonll((OVN_VXLAN_MIN_MULTICAST << 12));
            match_set_tun_id_masked(&match, mcast_bits, mcast_bits);

            put_load(1, MFF_LOG_OUTPORT, 15, 1, &ofpacts);
            put_move(MFF_TUN_ID, 12, MFF_LOG_OUTPORT,  0, 11, &ofpacts);
            put_move(MFF_TUN_ID, 0, MFF_LOG_DATAPATH, 0, 12, &ofpacts);
            put_resubmit(OFTABLE_LOCAL_OUTPUT, &ofpacts);

            ofctrl_add_flow(flow_table, OFTABLE_PHY_TO_LOG, 105, 0,
                            &match, &ofpacts, hc_uuid);
        }
    }
```

隧道占用的示意图

- geneve所使用的NXM_NX_TUN_METADATA0（mff_ovn_geneve）使用的低32位，NXM_NX_TUN_ID，使用的低24位。

```
NXM_NX_TUN_METADATA0：
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|rsv(1)|       inport(15)       |          output(16)         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
31    30                   16 15                         0

NXM_NX_TUN_ID：
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|             datapath(24)           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
23                              0
```

- stt所使用的NXM_NX_TUN_ID，使用的64位

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
| rsv(9) |       inport(15)       |          output(16)         |              datapath(24)           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
63      55 54                   40 39                         24 23                             0
```

- vxlan所使用的NXM_NX_TUN_ID，使用的低24位

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|     output(12)       |     datapath(12)     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

```
4  23              12 11              0
```

这里的output和datapath数据会放到vxlan协议中的vni字段中，因为vni的长度为24位，所以只保存了output和datapath。

发送和接收vxlan报文

发送vxlan报文例子

```
Frame 537: 148 bytes on wire (1184 bits), 148 bytes captured (1184 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Apr 26, 2025 17:59:07.645130000 CST
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1745661547.645130000 seconds
    [Time delta from previous captured frame: 0.009955000 seconds]
    [Time delta from previous displayed frame: 0.009955000 seconds]
    [Time since reference or first frame: 0.852829000 seconds]
    Frame Number: 537
    Frame Length: 148 bytes (1184 bits)
    Capture Length: 148 bytes (1184 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp:vxlan:eth:ethertype:ip:icmp:data]
Ethernet II, Src: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f), Dst: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c)
    Destination: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c)
        Address: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f)
        Address: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.20.2, Dst: 192.168.20.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 134
    Identification: 0x25eb (9707)
    Flags: 0x4000, Don't fragment
        0... .... .... .... = Reserved bit: Not set
        .1.. .... .... .... = Don't fragment: Set
        ..0. .... .... .... = More fragments: Not set
    Fragment offset: 0
    Time to live: 64
    Protocol: UDP (17)
    Header checksum: 0x6b26 [validation disabled]
    [Header checksum status: Unverified]
    Source: 192.168.20.2
    Destination: 192.168.20.3
User Datagram Protocol, Src Port: 38870, Dst Port: 4789
    Source Port: 38870
    Destination Port: 4789
    Length: 114
    Checksum: 0xa9d9 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 5]
```

```
    [Timestamps]
        [Time since first frame: 0.000000000 seconds]
        [Time since previous frame: 0.000000000 seconds]
Virtual eXtensible Local Area Network
    Flags: 0x0800, VXLAN Network ID (VNI)
        0... .... .... .... = GBP Extension: Not defined
        .... .... .0.. .... = Don't Learn: False
        .... 1... .... .... = VXLAN Network ID (VNI): True
        .... .... .... 0... = Policy Applied: False
        .000 .000 0.00 .000 = Reserved(R): 0x0000
    Group Policy ID: 0
    VXLAN Network Identifier (VNI): 20482
    Reserved: 0
Ethernet II, Src: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e), Dst: fa:16:3e:75:be:65 (fa:16:3e:75:be:65)
    Destination: fa:16:3e:75:be:65 (fa:16:3e:75:be:65)
        Address: fa:16:3e:75:be:65 (fa:16:3e:75:be:65)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e)
        Address: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.111.15, Dst: 192.168.111.69
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 84
    Identification: 0xdb54 (56148)
    Flags: 0x4000, Don't fragment
        0... .... .... .... = Reserved bit: Not set
        .1.. .... .... .... = Don't fragment: Set
        ..0. .... .... .... = More fragments: Not set
    Fragment offset: 0
    Time to live: 64
    Protocol: ICMP (1)
    Header checksum: 0xffae [validation disabled]
    [Header checksum status: Unverified]
    Source: 192.168.111.15
    Destination: 192.168.111.69
Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0x9327 [correct]
    [Checksum Status: Good]
    Identifier (BE): 22529 (0x5801)
    Identifier (LE): 344 (0x0158)
    Sequence number (BE): 331 (0x014b)
    Sequence number (LE): 19201 (0x4b01)
    Data (56 bytes)

[root@node-4 ~]# ovs-appctl dpctl/dump-flows --names|grep 0x29
recirc_id(0),in_port(tap8b09cd40-5d),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(src=192.168.111.15,proto=1,frag=no), packets:195, byt
recirc_id(0x29),in_port(tap8b09cd40-5d),ct_state(-new+est-rel-rpl+trk),ct_label(0/0x1),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(dst=1
[root@node-4 ~]#
[root@node-4 ~]#
```

```
[root@node-4 ~]#
[root@node-4 ~]# ovs-appctl ofproto/trace 'recirc_id(0),in_port(tap8b09cd40-5d),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(src=192.168
Flow: icmp,in_port=4,vlan_tci=0x0000,dl_src=fa:16:3e:c8:0b:4e,dl_dst=fa:16:3e:75:be:65,nw_src=192.168.111.15,nw_dst=0.0.0.0,nw_tos=0,nw_ecn=0,nw_ttl=0,icm

bridge("br-int")
----------------
 0. in_port=4, priority 100, cookie 0xee204cd9
    set_field:0x7->reg13
    set_field:0x6->reg11
    set_field:0x5->reg12
    set_field:0x2->metadata
    set_field:0x4->reg14
    resubmit(,8)
 8. reg14=0x4,metadata=0x2,dl_src=fa:16:3e:c8:0b:4e, priority 50, cookie 0xf54d2829
    resubmit(,9)
 9. ip,reg14=0x4,metadata=0x2,dl_src=fa:16:3e:c8:0b:4e,nw_src=192.168.111.15, priority 90, cookie 0x8ecff6f9
    resubmit(,10)
10. metadata=0x2, priority 0, cookie 0xcba1a41e
    resubmit(,11)
11. metadata=0x2, priority 0, cookie 0xff8e69bd
    resubmit(,12)
12. metadata=0x2, priority 0, cookie 0xa4ea953a
    resubmit(,13)
13. ip,metadata=0x2, priority 100, cookie 0xcb8e2ddf
    set_field:0x10000000000000000000000000/0x10000000000000000000000000->xxreg0
    resubmit(,14)
14. metadata=0x2, priority 0, cookie 0xf6e4120b
    resubmit(,15)
15. ip,reg0=0x1/0x1,metadata=0x2, priority 100, cookie 0xa21ed52f
    ct(table=16,zone=NXM_NX_REG13[0..15])
    drop
     -> A clone of the packet is forked to recirculate. The forked pipeline will be resumed at table 16.
     -> Sets the packet to an untracked state, and clears all the conntrack fields.

Final flow: icmp,reg0=0x1,reg11=0x6,reg12=0x5,reg13=0x7,reg14=0x4,metadata=0x2,in_port=4,vlan_tci=0x0000,dl_src=fa:16:3e:c8:0b:4e,dl_dst=fa:16:3e:75:be:65
Megaflow: recirc_id=0,eth,icmp,tun_id=0/0x800000,in_port=4,dl_src=fa:16:3e:c8:0b:4e,dl_dst=fa:16:3e:75:be:65,nw_src=192.168.111.15,nw_frag=no
Datapath actions: ct(zone=7),recirc(0x29)

===============================================================================
recirc(0x29) - resume conntrack with default ct_state=trk|new (use --ct-next to customize)
===============================================================================

Flow: recirc_id=0x29,ct_state=new|trk,ct_zone=7,eth,icmp,reg0=0x1,reg11=0x6,reg12=0x5,reg13=0x7,reg14=0x4,metadata=0x2,in_port=4,vlan_tci=0x0000,dl_src=f

bridge("br-int")
----------------
    thaw
        Resuming from table 16
16. ct_state=+new-est+trk,metadata=0x2, priority 7, cookie 0xd081458
    set_field:0x80000000000000000000000000/0x80000000000000000000000000->xxreg0
    set_field:0x20000000000000000000000000/0x20000000000000000000000000->xxreg0
    resubmit(,17)
17. ip,reg0=0x80/0x80,reg14=0x4,metadata=0x2, priority 2002, cookie 0x91e0b3dd
    set_field:0x20000000000000000000000000/0x20000000000000000000000000->xxreg0
    resubmit(,18)
18. metadata=0x2, priority 0, cookie 0x561f921b
    resubmit(,19)
19. metadata=0x2, priority 0, cookie 0xae803274
```

```
    resubmit(,20)
20. ip,reg0=0x2/0x2002,metadata=0x2, priority 100, cookie 0xe3cba6b8
    ct(commit,zone=NXM_NX_REG13[0..15],nat(src),exec(set_field:0/0x1->ct_label))
    nat(src)
    set_field:0/0x1->ct_label
     -> Sets the packet to an untracked state, and clears all the conntrack fields.
    resubmit(,21)
21. metadata=0x2, priority 0, cookie 0x7ef6d074
    resubmit(,22)
22. metadata=0x2, priority 0, cookie 0x87ed3740
    resubmit(,23)
23. metadata=0x2, priority 0, cookie 0x75a38d4a
    resubmit(,24)
24. metadata=0x2, priority 0, cookie 0xaebf9b38
    resubmit(,25)
25. metadata=0x2, priority 0, cookie 0xc567b5bc
    resubmit(,26)
26. metadata=0x2, priority 0, cookie 0x1b63ec86
    resubmit(,27)
27. metadata=0x2, priority 0, cookie 0x4869075b
    resubmit(,28)
28. metadata=0x2, priority 0, cookie 0x13470042
    resubmit(,29)
29. metadata=0x2, priority 0, cookie 0x612cf8a
    resubmit(,30)
30. metadata=0x2,dl_dst=fa:16:3e:75:be:65, priority 50, cookie 0x4a96246
    set_field:0x5->reg15
    resubmit(,37)
37. reg15=0x5,metadata=0x2, priority 100, cookie 0x54962c2d
    set_field:0x5002/0xffffff->tun_id
    output:6
     -> output to kernel tunnel

Final flow: recirc_id=0x29,eth,icmp,reg0=0x283,reg11=0x6,reg12=0x5,reg13=0x7,reg14=0x4,reg15=0x5,tun_id=0x5002,metadata=0x2,in_port=4,vlan_tci=0x0000,d
Megaflow: recirc_id=0x29,ct_state=+new-est-rel-rpl+trk,ct_label=0/0x1,eth,icmp,tun_id=0/0xffffff,in_port=4,dl_src=fa:16:3e:c8:0b:4e,dl_dst=fa:16:3e:75:be:65,nw_
Datapath actions: ct(commit,zone=7,label=0/0x1,nat(src)),set(tunnel(tun_id=0x5002,dst=192.168.20.3,ttl=64,tp_dst=4789,flags(df|csum|key))),27

cookie=0x54962c2d, duration=76881.043s, table=37, n_packets=73132, n_bytes=7166936, priority=100,reg15=0x5,metadata=0x2 actions=load:0x5002->NXM_NX_

这里0x5002等于20482

```

发送vxlan报文的ovs源码

通过分析以下的openflow和datapath流表来查看发送vxlan报文的ovs源码流程

```
cookie=0x54962c2d, duration=76881.043s, table=37, n_packets=73132, n_bytes=7166936, priority=100,reg15=0x5,metadata=0x2 actions=load:0x5002->NXM_NX_TU
recirc_id(0x29),in_port(tap8b09cd40-5d),ct_state(-new+est-rel-rpl+trk),ct_label(0/0x1),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(dst=192
```

```
recv_upcalls
 |- process_upcall
  |- case MISS_UPCALL:
   |- upcall_xlate(udpif, upcall, odp_actions, wc);
    |- xerr = xlate_actions(&xin, &upcall->xout);
     |- rule_dpif_lookup_from_table // 流表查找
     |- do_xlate_actions
      |- case OFPACT_SET_FIELD:
       |- mf_set_flow_value_masked
```

```
      |- mf_set_flow_value(field, &tmp, flow);
       |- case MFF_TUN_ID:
        |- flow->tunnel.tun_id = value->be64; // 执行这个action：load:0x5002->NXM_NX_TUN_ID[0..23]
    // 上面的load action会将tun_id设置到flow->tunnel中
    |- case OFPACT_OUTPUT:
     |- xlate_output_action
      |- compose_output_action(ctx, port, NULL, is_last_action, truncate);
       |- compose_output_action__
        |- if (xport->is_tunnel)
         |- odp_port = tnl_port_send(xport->ofport, flow, ctx->wc);
          |- tnl_port = tnl_find_ofport(ofport); // 根据openflow port找到datapath port，他们之间有对应关系，通过命令`ovs-appctl dpif/show`查看
          |- out_port = tnl_port ? tnl_port->match.odp_port : ODPP_NONE;
          |- out_port = odp_port; // 将out_port设置为datapath port。
          |- commit_odp_tunnel_action
           |- odp_put_tunnel_action(&base->tunnel, odp_actions, tnl_type);
            |- size_t offset = nl_msg_start_nested(odp_actions, OVS_ACTION_ATTR_SET); //OVS_ACTION_ATTR_SET
            |- tun_key_to_attr(odp_actions, tunnel, tunnel, NULL, tnl_type); // 这里会将flow->tunnel中的数据解析出来设置为datapath actions:set(tunnel(tun_id=0x500
             |- tun_key_ofs = nl_msg_start_nested(a, OVS_KEY_ATTR_TUNNEL);
             |- if (tun_key->tun_id || tun_key->flags & FLOW_TNL_F_KEY)
              |- nl_msg_put_be64(a, OVS_TUNNEL_KEY_ATTR_ID, tun_key->tun_id);
         |- if (out_port != ODPP_NONE)
          |- nl_msg_put_odp_port(ctx->odp_actions, OVS_ACTION_ATTR_OUTPUT, out_port); // 执行这个action：vxlan_sys_4789
```

```
ovs_packet_cmd_execute
 |- ovs_nla_copy_actions
  |- __ovs_nla_copy_actions
   |- case OVS_ACTION_ATTR_SET:
    |- validate_set
     |- case OVS_KEY_ATTR_TUNNEL:
      |- err = validate_and_copy_set_tun(a, sfa, log);
       |- opts_type = ip_tun_from_nlattr(nla_data(attr), &match, false, log);
        |- case OVS_TUNNEL_KEY_ATTR_ID:
         |- SW_FLOW_KEY_PUT(match, tun_key.tun_id, nla_get_be64(a), is_mask);
         |- tun_flags |= TUNNEL_KEY;
      |- start = add_nested_action_start(sfa, OVS_ACTION_ATTR_SET, log);
      |- a = __add_action(sfa, OVS_KEY_ATTR_TUNNEL_INFO, NULL, sizeof(*ovs_tun), log); // 将tun_info放到OVS_ACTION_ATTR_SET->OVS_KEY_ATTR_TU
|- err = ovs_execute_actions(dp, packet, sf_acts, &flow->key);
 |- do_execute_actions
  |- case OVS_ACTION_ATTR_SET:
   |- err = execute_set_action(skb, key, nla_data(a));
    |- if (nla_type(a) == OVS_KEY_ATTR_TUNNEL_INFO)
     |- ovs_skb_dst_set(skb, (struct dst_entry *)tun->tun_dst); // #define ovs_skb_dst_set skb_dst_set;会调用内核函数：skb_dst_set
      |- skb->_skb_refdst = (unsigned long)dst;
  |- case OVS_ACTION_ATTR_OUTPUT:
   |- do_output(dp, clone, port, key);
    |- ovs_vport_send(vport, skb, ovs_key_mac_proto(key));
     |- vport->ops->send(skb);
      |- dev_queue_xmit //vport-netdev.c中的send函数。
```

```
dev_queue_xmit
 |- __dev_queue_xmit(skb, NULL);
  |- dev_hard_start_xmit(skb, dev, txq, &rc);
   |- xmit_one(skb, dev, txq, next != NULL);
    |- if (dev_nit_active(dev))
     |- dev_queue_xmit_nit(skb, dev); //tcpdump抓包
    |- netdev_start_xmit(skb, dev, txq, more);
     |- __netdev_start_xmit(ops, skb, dev, more);
      |- ops->ndo_start_xmit(skb, dev); //调用对应网络设备的hook函数。这里会调用到内核中vxlan.c中的ndo_start_xmit函数，即vxlan_xmit函数
```

```
        |- info = skb_tunnel_info(skb);
         |- struct metadata_dst *md_dst = skb_metadata_dst(skb);
          |- struct metadata_dst *md_dst = (struct metadata_dst *) skb_dst(skb); // 获取skb->_skb_refdst
        |- vni = tunnel_id_to_key32(info->key.tun_id); // 获取到tun_id，后续会设置到vxlan协议的vni字段中。
```

接收vxlan报文例子

```

Frame 538: 148 bytes on wire (1184 bits), 148 bytes captured (1184 bits)
  Encapsulation type: Ethernet (1)
  Arrival Time: Apr 26, 2025 17:59:07.648206000 CST
  [Time shift for this packet: 0.000000000 seconds]
  Epoch Time: 1745661547.648206000 seconds
  [Time delta from previous captured frame: 0.003076000 seconds]
  [Time delta from previous displayed frame: 0.003076000 seconds]
  [Time since reference or first frame: 0.855905000 seconds]
  Frame Number: 538
  Frame Length: 148 bytes (1184 bits)
  Capture Length: 148 bytes (1184 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:udp:vxlan:eth:ethertype:ip:icmp:data]
Ethernet II, Src: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c), Dst: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f)
  Destination: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f)
    Address: ce:c9:1c:f8:83:4f (ce:c9:1c:f8:83:4f)
    .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
    .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  Source: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c)
    Address: 96:eb:92:a7:c9:4c (96:eb:92:a7:c9:4c)
    .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
    .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.20.3, Dst: 192.168.20.2
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 134
  Identification: 0x0215 (533)
  Flags: 0x4000, Don't fragment
    0... .... .... .... = Reserved bit: Not set
    .1.. .... .... .... = Don't fragment: Set
    ..0. .... .... .... = More fragments: Not set
  Fragment offset: 0
  Time to live: 64
  Protocol: UDP (17)
  Header checksum: 0x8efc [validation disabled]
  [Header checksum status: Unverified]
  Source: 192.168.20.3
  Destination: 192.168.20.2
User Datagram Protocol, Src Port: 41764, Dst Port: 4789
  Source Port: 41764
  Destination Port: 4789
  Length: 114
  Checksum: 0x527b [unverified]
  [Checksum Status: Unverified]
  [Stream index: 6]
```

```
    [Timestamps]
        [Time since first frame: 0.000000000 seconds]
        [Time since previous frame: 0.000000000 seconds]
Virtual eXtensible Local Area Network
    Flags: 0x0800, VXLAN Network ID (VNI)
        0... .... .... .... = GBP Extension: Not defined
        .... .... .0.. .... = Don't Learn: False
        .... 1... .... .... = VXLAN Network ID (VNI): True
        .... .... .... 0... = Policy Applied: False
        .000 .000 0.00 .000 = Reserved(R): 0x0000
    Group Policy ID: 0
    VXLAN Network Identifier (VNI): 16386
    Reserved: 0
Ethernet II, Src: fa:16:3e:75:be:65 (fa:16:3e:75:be:65), Dst: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e)
    Destination: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e)
        Address: fa:16:3e:c8:0b:4e (fa:16:3e:c8:0b:4e)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: fa:16:3e:75:be:65 (fa:16:3e:75:be:65)
        Address: fa:16:3e:75:be:65 (fa:16:3e:75:be:65)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.111.69, Dst: 192.168.111.15
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 84
    Identification: 0xbf21 (48929)
    Flags: 0x0000
        0... .... .... .... = Reserved bit: Not set
        .0.. .... .... .... = Don't fragment: Not set
        ..0. .... .... .... = More fragments: Not set
    Fragment offset: 0
    Time to live: 64
    Protocol: ICMP (1)
    Header checksum: 0x5be2 [validation disabled]
    [Header checksum status: Unverified]
    Source: 192.168.111.69
    Destination: 192.168.111.15
Internet Control Message Protocol
    Type: 0 (Echo (ping) reply)
    Code: 0
    Checksum: 0x9b27 [correct]
    [Checksum Status: Good]
    Identifier (BE): 22529 (0x5801)
    Identifier (LE): 344 (0x0158)
    Sequence number (BE): 331 (0x014b)
    Sequence number (LE): 19201 (0x4b01)
    [Request frame: 537]
    [Response time: 3.076 ms]
    Data (56 bytes)

[root@node-4 ~]# ovs-appctl dpctl/dump-flows --names|grep 192.168.111
recirc_id(0x2a),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,flags(-df+csum+key)),in_port(vxlan_sys_4789),ct_state(-new+est-rel+rpl+trk),ct_label(0/0x
recirc_id(0x29),in_port(tap8b09cd40-5d),ct_state(-new+est-rel-rpl+trk),ct_label(0/0x1),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(dst=1
```

```
recirc_id(0),in_port(tap8b09cd40-5d),eth(src=fa:16:3e:c8:0b:4e,dst=fa:16:3e:75:be:65),eth_type(0x0800),ipv4(src=192.168.111.15,proto=1,frag=no), packets:18, bytes
[root@node-4 ~]# ovs-appctl dpctl/dump-flows --names|grep 0x2a
recirc_id(0),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,flags(-df+csum+key)),in_port(vxlan_sys_4789),eth(src=fa:16:3e:75:be:65,dst=00:00:00:00:00:0
recirc_id(0x2a),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,flags(-df+csum+key)),in_port(vxlan_sys_4789),ct_state(-new+est-rel+rpl+trk),ct_label(0/0x
[root@node-4 ~]#
[root@node-4 ~]#
[root@node-4 ~]# ovs-appctl ofproto/trace 'recirc_id(0),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,ttl=64,flags(-df+csum+key)),in_port(vxlan_sys_47
Flow: icmp,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_ipv6_src=::,tun_ipv6_dst=::,tun_gbp_id=0,tun_gbp_flags=0,tun_tos=0,tun_ttl=64,tun_ers

bridge("br-int")
----------------
 0. in_port=6, priority 100
    move:NXM_NX_TUN_ID[12..23]->NXM_NX_REG15[0..11]
     -> NXM_NX_REG15[0..11] is now 0x4
    move:NXM_NX_TUN_ID[0..11]->OXM_OF_METADATA[0..11]
     -> OXM_OF_METADATA[0..11] is now 0x2
    resubmit(,38)
38. reg15=0x4,metadata=0x2, priority 100, cookie 0xee204cd9
    set_field:0x7->reg13
    set_field:0x6->reg11
    set_field:0x5->reg12
    resubmit(,39)
39. priority 0
    set_field:0->reg0
    set_field:0->reg1
    set_field:0->reg2
    set_field:0->reg3
    set_field:0->reg4
    set_field:0->reg5
    set_field:0->reg6
    set_field:0->reg7
    set_field:0->reg8
    set_field:0->reg9
    resubmit(,40)
40. metadata=0x2, priority 0, cookie 0xe6a44197
    resubmit(,41)
41. ip,metadata=0x2, priority 100, cookie 0x5bb4e126
    set_field:0x10000000000000000000000000/0x10000000000000000000000000->xxreg0
    resubmit(,42)
42. ip,reg0=0x1/0x1,metadata=0x2, priority 100, cookie 0x6bf56d91
    ct(table=43,zone=NXM_NX_REG13[0..15])
    drop
     -> A clone of the packet is forked to recirculate. The forked pipeline will be resumed at table 43.
     -> Sets the packet to an untracked state, and clears all the conntrack fields.

Final flow: icmp,reg0=0x1,reg11=0x6,reg12=0x5,reg13=0x7,reg15=0x4,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_ipv6_src=::,tun_ipv6_dst=::,
Megaflow: recirc_id=0,eth,icmp,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_tos=0,tun_flags=-df+csum+key,in_port=6,dl_src=fa:16:3e:75:be:65,d
Datapath actions: ct(zone=7),recirc(0x2b)

===============================================================================
recirc(0x2b) - resume conntrack with default ct_state=trk|new (use --ct-next to customize)
===============================================================================

Flow: recirc_id=0x2b,ct_state=new|trk,ct_zone=7,eth,icmp,reg0=0x1,reg11=0x6,reg12=0x5,reg13=0x7,reg15=0x4,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.

bridge("br-int")
----------------
    thaw
```

```
        Resuming from table 43
43. ct_state=+new-est+trk,metadata=0x2, priority 7, cookie 0xfffa9c2d
    set_field:0x80000000000000000000000000000000/0x80000000000000000000000000000000->xxreg0
    set_field:0x20000000000000000000000000000000/0x20000000000000000000000000000000->xxreg0
    resubmit(,44)
44. ip,reg0=0x200/0x200,reg15=0x4,metadata=0x2, priority 2001, cookie 0x5db9564f
    drop

Final flow: recirc_id=0x2b,ct_state=new|trk,ct_zone=7,eth,icmp,reg0=0x281,reg11=0x6,reg12=0x5,reg13=0x7,reg15=0x4,tun_id=0x4002,tun_src=192.168.20.3,tun_c
Megaflow: recirc_id=0x2b,ct_state=+new-est-rel-rpl+trk,ct_label=0/0x1,eth,ip,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_tos=0,tun_flags=-df+c
Datapath actions: drop
[root@node-4 ~]#
[root@node-4 ~]# ovs-appctl ofproto/trace 'recirc_id(0x2a),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,ttl=64,flags(-df+csum+key)),in_port(vxlan_sys
Flow: recirc_id=0x2a,ct_state=est|rpl|trk,eth,ip,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_ipv6_src=::,tun_ipv6_dst=::,tun_gbp_id=0,tun_gbp_fl

bridge("br-int")
----------------
    thaw
        Resuming from table 43
43. ct_state=+est+trk,ct_label=0/0x1,metadata=0x2, priority 1, cookie 0x876d9760
    set_field:0x40000000000000000000000000000000/0x40000000000000000000000000000000->xxreg0
    resubmit(,44)
44. ct_state=-new+est-rel+rpl+trk,ct_label=0/0x1,metadata=0x2, priority 65532, cookie 0x682777e8
    resubmit(,45)
45. metadata=0x2, priority 0, cookie 0x49d1a0d3
    resubmit(,46)
46. metadata=0x2, priority 0, cookie 0xe6a56678
    resubmit(,47)
47. metadata=0x2, priority 0, cookie 0x7c2f99da
    resubmit(,48)
48. ip,reg15=0x4,metadata=0x2,dl_dst=fa:16:3e:c8:0b:4e,nw_dst=192.168.111.15, priority 90, cookie 0xdb009641
    resubmit(,49)
49. reg15=0x4,metadata=0x2,dl_dst=fa:16:3e:c8:0b:4e, priority 50, cookie 0xf59d6ebd
    resubmit(,64)
64. priority 0
    resubmit(,65)
65. reg15=0x4,metadata=0x2, priority 100, cookie 0xee204cd9
    output:4

Final flow: recirc_id=0x2a,ct_state=est|rpl|trk,eth,ip,reg0=0x401,reg11=0x6,reg12=0x5,reg13=0x7,reg15=0x4,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.
Megaflow: recirc_id=0x2a,ct_state=-new+est-rel+rpl+trk,ct_label=0/0x1,eth,ip,tun_id=0x4002,tun_src=192.168.20.3,tun_dst=192.168.20.2,tun_tos=0,tun_flags=-df+c
Datapath actions: 25
[root@node-4 ~]#

cookie=0x0, duration=76881.042s, table=0, n_packets=160804, n_bytes=12953032, priority=100,in_port="ovn-4c4edd-0" actions=move:NXM_NX_TUN_ID[12..23]


这里0x4002等于16386
```

接收vxlan报文的ovs源码

通过分析以下的openflow和datapath流表来查看接收vxlan报文的ovs源码流程

```
cookie=0x0, duration=76881.042s, table=0, n_packets=160804, n_bytes=12953032, priority=100,in_port="ovn-4c4edd-0" actions=move:NXM_NX_TUN_ID[12..23]->
recirc_id(0),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,flags(-df+csum+key)),in_port(vxlan_sys_4789),eth(src=fa:16:3e:75:be:65,dst=00:00:00:00:00:00/
recirc_id(0x2a),tunnel(tun_id=0x4002,src=192.168.20.3,dst=192.168.20.2,flags(-df+csum+key)),in_port(vxlan_sys_4789),ct_state(-new+est-rel+rpl+trk),ct_label(0/0x1)
```

```
netdev_frame_hook
```

```
|- netdev_port_receive(skb, skb_tunnel_info(skb));
 |- skb_tunnel_info(skb)
  |- struct metadata_dst *md_dst = skb_metadata_dst(skb);
   |- struct metadata_dst *md_dst = (struct metadata_dst *) skb_dst(skb);
  |- return &md_dst->u.tun_info;
 |- ovs_vport_receive(vport, skb, tun_info);
  |- error = ovs_flow_key_extract(tun_info, skb, &key); // 解析skb，设置到sw_flow_key中。
   |- if (tun_info)
    |- memcpy(&key->tun_key, &tun_info->key, sizeof(key->tun_key)); // 将tun_info复制到sw_flow_key的tun_key中。
   |- err = key_extract(skb, key); // 主要的将skb的数据解析到sw_flow_key中，比如2,3,4层的数据。
    |- return key_extract_l3l4(skb, key);
  |- ovs_dp_process_packet(skb, &key);
   |- error = ovs_dp_upcall(dp, skb, key, &upcall, 0); //如果在内核中没找到相应的流表，则upcall到userspace进行处理。
    |- err = queue_userspace_packet(dp, skb, key, upcall_info, cutlen);
     |- user_skb = genlmsg_new(len, GFP_ATOMIC); // 创建了一个user_skb,然后将sw_flow_key的数据设置到user_skb的OVS_PACKET_ATTR_KEY属性中。
     |- err = ovs_nla_put_key(key, key, OVS_PACKET_ATTR_KEY, false, user_skb);
      |- err = __ovs_nla_put_key(swkey, output, is_mask, skb);
       |- if ((swkey->tun_proto || is_mask))
        |- ip_tun_to_nlattr
         |- nla = nla_nest_start_noflag(skb, OVS_KEY_ATTR_TUNNEL);
         |- err = __ip_tun_to_nlattr(skb, output, tun_opts, swkey_tun_opts_len, tun_proto);
          |- nla_put_be64(skb, OVS_TUNNEL_KEY_ATTR_ID, output->tun_id, OVS_TUNNEL_KEY_ATTR_PAD) // 将tun_id设置到ovs_key中。
     |- nla_reserve(user_skb, OVS_PACKET_ATTR_PACKET, 0)
     |- err = skb_zerocopy(user_skb, skb, skb->len - cutlen, hlen); // 把原始报文的skb拷贝也放到到user_skb的OVS_PACKET_ATTR_PACKET属性中。
```

```
recv_upcalls
 |- dpif_recv(udpif->dpif, handler->handler_id, dupcall, recv_buf)
  |- if (dpif->dpif_class->recv)
   |- error = dpif->dpif_class->recv(dpif, handler_id, upcall, buf);
    |- dpif_netlink_recv // dpif-netlink.c
     |- if (dpif_netlink_upcall_per_cpu(dpif))
      |- error = dpif_netlink_recv_cpu_dispatch(dpif, handler_id, upcall, buf);
       |- error = parse_odp_packet(buf, upcall, &dp_ifindex);
        |- nl_policy_parse(&b, 0, ovs_packet_policy, a, ARRAY_SIZE(ovs_packet_policy)) // 解析netlink从kernel到userspace中的数据，解析的数据放到upcall变量中
        |- upcall->key = CONST_CAST(struct nlattr *, nl_attr_get(a[OVS_PACKET_ATTR_KEY]));
        |- upcall->userdata = a[OVS_PACKET_ATTR_USERDATA];
        |- upcall->hash = a[OVS_PACKET_ATTR_HASH];
 |- upcall->fitness = odp_flow_key_to_flow(dupcall->key, dupcall->key_len, flow, NULL); // 将upcall->key进行解析，解析到flow中。这里设置的flow字段都是从s
  |- odp_flow_key_to_flow__(key, key_len, flow, flow, errorp);
   |- if (present_attrs & (UINT64_C(1) << OVS_KEY_ATTR_TUNNEL))
    |- res = odp_tun_key_from_attr__(attrs[OVS_KEY_ATTR_TUNNEL], is_mask, &flow->tunnel, errorp);
     |- case OVS_TUNNEL_KEY_ATTR_ID:
      |- tun->tun_id = nl_attr_get_be64(a);
      |- tun->flags |= FLOW_TNL_F_KEY;
 |- process_upcall
  |- case MISS_UPCALL:
   |- upcall_xlate(udpif, upcall, odp_actions, wc);
    |- xerr = xlate_actions(&xin, &upcall->xout);
     |- rule_dpif_lookup_from_table // 流表查找，这里是根据flow字段的值，从第一个openflow流表开始查找，找到对应的流表后，向下执行action。
     |- do_xlate_actions(ofpacts, ofpacts_len, &ctx, true, false); // 在设置action(非output的action)时，会再次设置flow的一些字段（这里是根据openflow的action设置
      |- case OFPACT_REG_MOVE:
       |- xlate_ofpact_reg_move(ctx, ofpact_get_REG_MOVE(a)); // 执行这些action：move:NXM_NX_TUN_ID[12..23]->NXM_NX_REG15[0..11],move:NXM_NX_TU
        |- mf_subfield_copy(&a->src, &a->dst, &ctx->xin->flow, ctx->wc);
         |- mf_get_value(dst->field, flow, &dst_value); // 获取move的dst数据的值。
         |- mf_get_value(src->field, flow, &src_value); // 获取move的src数据的值。
          |- case MFF_TUN_ID:
           |- value->be64 = flow->tunnel.tun_id;
        |- mf_set_flow_value(dst->field, &dst_value, flow);
```

```
34          |- case MFF_METADATA:
35           |- flow->metadata = value->be64; // 将NXM_NX_TUN_ID[0..11]的值设置到OXM_OF_METADATA[0..11]中
36          |- CASE_MFF_REGS:
37           |- flow->regs[mf->id - MFF_REG0] = ntohl(value->be32); // 将NXM_NX_TUN_ID[12..23]的值设置到NXM_NX_REG15[0..11]中
38        |- case OFPACT_RESUBMIT:
39         |- xlate_ofpact_resubmit(ctx, ofpact_get_RESUBMIT(a), last); // 执行这个action：resubmit(,38)，执行resubmit时，又会继续openflow流表且执行相应的action
```

总结

- ovs内核收到packet，将packet的数据解析到sw_flow_key中，然后将sw_flow_key的数据设置到user_skb中，然后通过netlink将user_skb发送到userspace。
- 在ovs的userspace中，将user_skb的数据解析到upcall中，然后将upcall中的key（struct nlattr*）的数据解析到flow中.这里设置的flow字段都是从skb原始报文中解析出来的。
- 然后执行openflow流表查找，找到对应的openflow流表后，将对应的非output的action设置到flow字段中。
- 可能多次执行步骤3（比如有resubmit action），直到找到最终的output action。